

Học Máy (Machine Learning)

Chương 8: Giảm chiều dữ liệu (Dimensionality Reduction)

Giảng viên: TS. Trần Thành Thắng

Đại học Đông Á

August 5, 2026

Nội dung Chương trình

- 1 Khái quát về Giảm chiều dữ liệu
- 2 Phân tích thành phần chính (PCA)
- 3 Phép chiếu ngẫu nhiên & LLE

Giới thiệu Giảm chiều dữ liệu

- Trong thực tế, nhiều bài toán học máy liên quan đến hàng nghìn hoặc hàng triệu đặc trưng (features) cho mỗi trường hợp huấn luyện.
- Việc này không chỉ làm quá trình huấn luyện cực kỳ chậm mà còn làm cho việc tìm ra một giải pháp tốt trở nên khó khăn hơn.
- Vấn đề này thường được gọi là **lời nguyền của số chiều** (the curse of dimensionality).
- Kỹ thuật giảm chiều (Dimensionality Reduction) giúp biến bài toán phức tạp thành bài toán có thể giải quyết được bằng cách loại bỏ các đặc trưng không quan trọng hoặc dư thừa.

Lợi ích 1: Tăng tốc quá trình huấn luyện

- Giảm số lượng đặc trưng giúp thuật toán học máy chạy nhanh hơn đáng kể, tốn ít bộ nhớ hơn.
- **Ví dụ (Ảnh MNIST):** Các pixel ở đường biên hình ảnh hầu như luôn trắng.
- Ta có thể loại bỏ hoàn toàn các pixel này khỏi tập huấn luyện mà không làm mất nhiều thông tin hữu ích cho tác vụ phân loại chữ số.
- Hai pixel lân cận thường có tương quan cao, có thể hợp nhất chúng (ví dụ: lấy trung bình) để giảm dữ liệu.

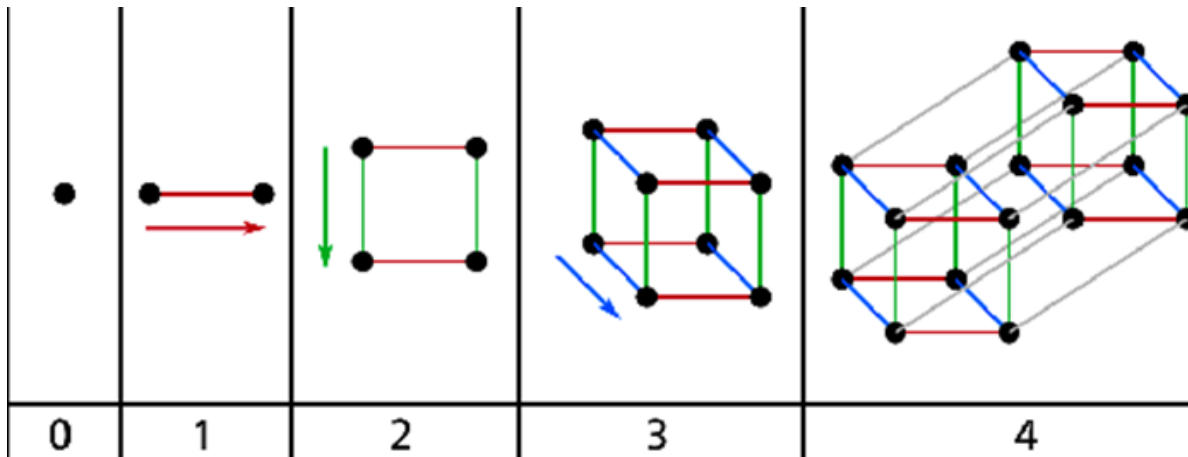
Lợi ích 2: Trực quan hóa dữ liệu

- Trực quan hóa là điều cần thiết để truyền đạt kết luận của nhà khoa học dữ liệu cho những người ra quyết định.
- Rất khó để vẽ một đồ thị có hơn 3 chiều.
- Giảm số lượng chiều xuống còn 2 hoặc 3 giúp vẽ biểu đồ trực quan, giúp con người dễ dàng quan sát và phát hiện các mẫu (chẳng hạn như các cụm/clusters).

Lời nguyên của số chiều

- Chúng ta quen sống trong không gian 3D, do đó trực giác thường thất bại khi hình dung không gian có chiều cao (ví dụ: 1.000 chiều).
- Trong không gian chiều cao, nhiều thứ hoạt động rất khác.
- Một điểm ngẫu nhiên trong hình vuông 1×1 (2D) chỉ có 0.4% cơ hội nằm cách đường biên dưới 0.001.
- Trong một siêu hình lập phương 10.000 chiều, xác suất này $> 99.999999\%$.
- **Kết luận:** Hầu hết các điểm trong một không gian có chiều cao đều nằm *rất gần đường biên*.

Hình ảnh: Lời nguyên số chiều



Hình 8-1. Điểm, đoạn thẳng, hình vuông, hình lập phương và tesseract (siêu hình lập

Sự khác biệt khoảng cách trong không gian chiều cao

- Nếu chọn hai điểm ngẫu nhiên trong một hình vuông 2D đơn vị, khoảng cách trung bình là ≈ 0.52 .
- Trong hình lập phương 3D đơn vị, khoảng cách trung bình là ≈ 0.66 .
- Trong một siêu hình lập phương đơn vị 1.000.000 chiều, khoảng cách trung bình là ≈ 408.25 .
- Có rất nhiều không gian trong các chiều cao, dẫn đến dữ liệu vô cùng thưa thớt (sparse).

Nguy cơ quá khớp do thừa thớt

- Tập dữ liệu có chiều cao $\rightarrow$ hầu hết các trường hợp huấn luyện cách rất xa nhau.
- Một trường hợp mới (test instance) có thể sẽ cách xa bất kỳ trường hợp huấn luyện nào đã biết.
- Dự đoán sẽ kém tin cậy hơn nhiều vì phải dựa trên các phép *ngoại suy* (*extrapolation*) lớn.
- **Tóm lại:** Tập huấn luyện càng có nhiều chiều, nguy cơ **quá khớp (overfitting)** càng lớn.

Các cách tiếp cận chính để giảm chiều

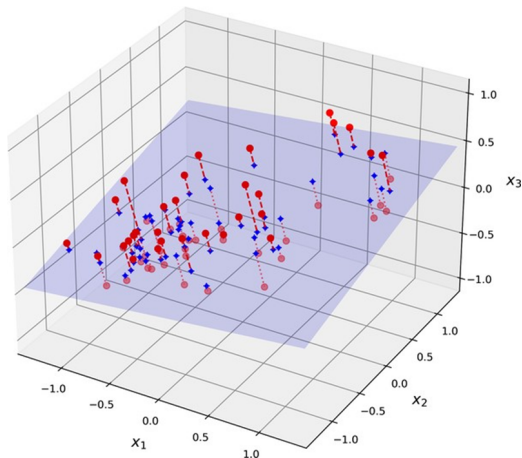
Thay vì cố gắng thu thập dữ liệu bằng số nguyên tử vũ trụ (để đủ độ dày đặc), chúng ta dùng các thuật toán giảm chiều:

- 1 **Phép chiếu (Projection)**
- 2 **Học đa tạp (Manifold Learning)**

Cách tiếp cận 1: Phép chiếu (Projection)

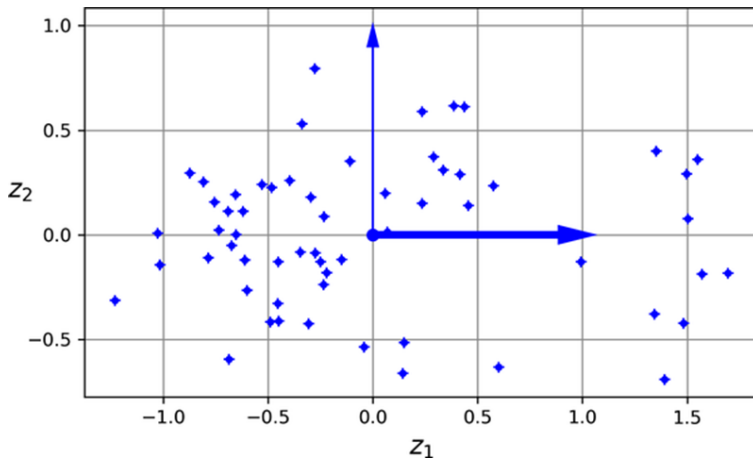
- Trong hầu hết các bài toán thực tế, các trường hợp huấn luyện không được phân bố đều đặn (uniform) trên tất cả các chiều.
- Nhiều đặc trưng gần như không đổi, trong khi các đặc trưng khác tương quan mạnh.
- Kết quả là, mọi trường hợp huấn luyện thường nằm gọn trong (hoặc rất gần) một **không gian con (subspace) có số chiều thấp hơn nhiều** so với không gian gốc.

Tập dữ liệu 3D gần không gian con 2D



Hình 8-2. Một tập dữ liệu 3D nằm gần một không gian con 2D (mặt phẳng)

Tập dữ liệu 2D mới sau phép chiếu

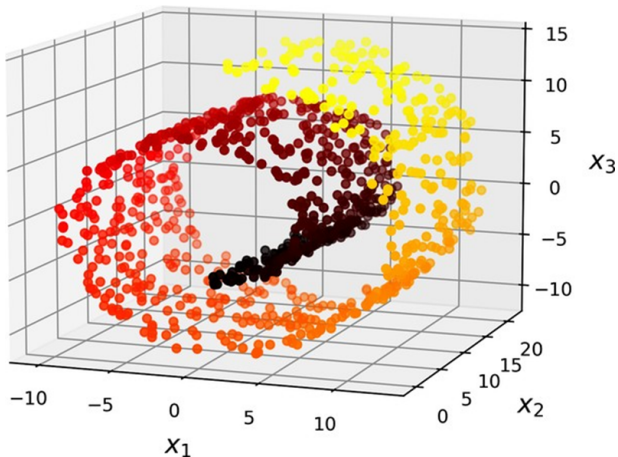


Hình 8-3. Bằng cách chiếu vuông góc các điểm lên mặt phẳng, ta thu được tập dữ liệu 2D mới

Cách tiếp cận 2: Học đa tạp (Manifold Learning)

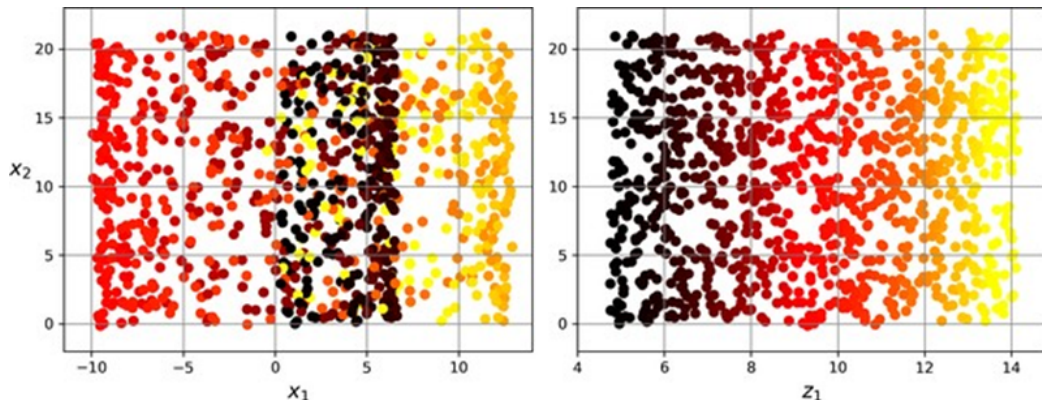
- Phép chiếu không phải lúc nào cũng là cách tiếp cận tốt nhất.
- Trong nhiều trường hợp, không gian con có thể bị **xoắn và uốn cong**.
- Ví dụ điển hình: Tập dữ liệu **Swiss roll**.

Tập dữ liệu Swiss roll



Hình 8-4. Tập dữ liệu Swiss roll (như một tờ giấy bị cuộn lại trong không gian 3D)

Làm dẹt (Phép chiếu) vs Mở cuộn (Đa tạp)

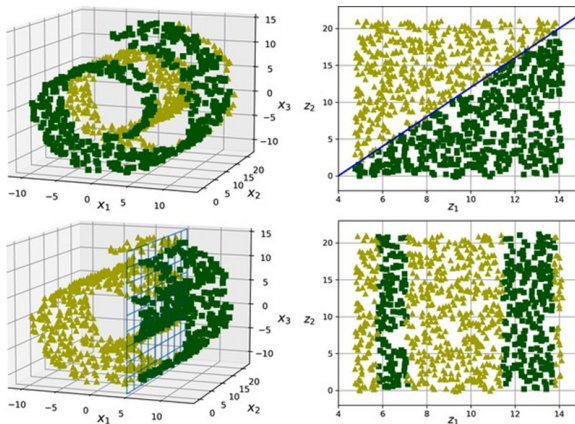


Hình 8-5. Chiếu vuông góc làm dẹt cuộn (trái) - sai lầm; Mở cuộn (phải) - chính xác

Giả định đa tạp (Manifold Hypothesis)

- **Đa tạp 2D:** là một hình 2D có thể bị uốn cong, xoắn lại trong một không gian có chiều cao hơn. Tổng quát, một đa tạp d -chiều nằm trong không gian n -chiều ($d < n$).
- **Giả định đa tạp:** Hầu hết các tập dữ liệu thực tế có chiều cao đều nằm gần một đa tạp có chiều thấp hơn nhiều.
- Các tác vụ học máy (như phân loại/hồi quy) **thường** sẽ đơn giản hơn nếu được biểu diễn trong không gian đa tạp (sau khi được mở cuộn).

Lưu ý về Giả định đa tạp



Hình 8-6. Đường biên quyết định không phải lúc nào cũng đơn giản hơn với không gian chiều thấp

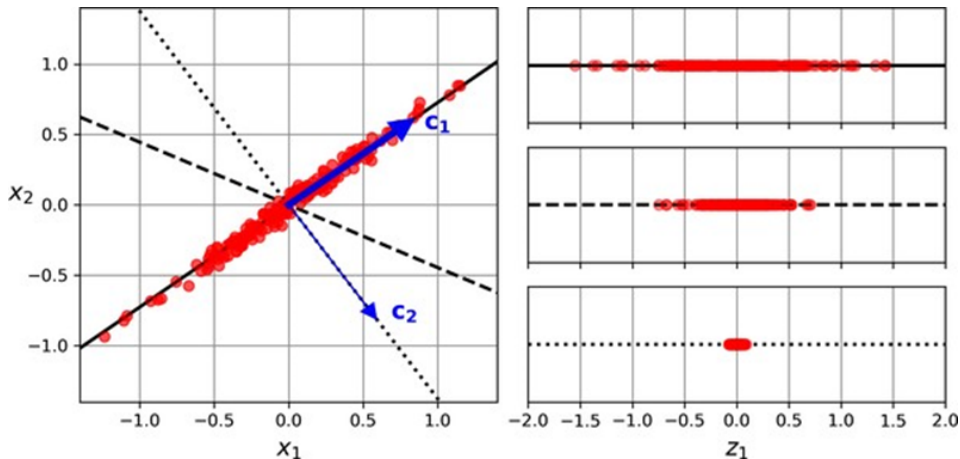
Phân tích thành phần chính (PCA)

- **Phân tích thành phần chính (Principal Component Analysis - PCA)** cho đến nay là thuật toán giảm chiều phổ biến nhất.
- Nó là một phương pháp thuộc nhóm **Phép chiếu (Projection)**.
- Cơ chế: Đầu tiên xác định siêu phẳng (hyperplane) nằm gần dữ liệu nhất, sau đó chiếu toàn bộ dữ liệu lên mặt phẳng đó.

Bảo toàn phương sai (Preserving Variance)

- Trước khi chiếu dữ liệu, cần phải **chọn một siêu phẳng phù hợp**.
- PCA sẽ chọn siêu phẳng (hoặc trục) sao cho **bảo toàn được lượng phương sai lớn nhất** của dữ liệu.
- Việc này giúp giữ lại được nhiều thông tin nhất có thể sau phép chiếu.
- Nó cũng tương đương với việc chọn trục làm *giảm thiểu khoảng cách bình phương trung bình* giữa tập dữ liệu gốc và điểm chiếu.

Lựa chọn không gian con để chiếu



Hình 8-7. Phép chiếu lên đường liền nét bảo toàn phương sai tối đa

Các thành phần chính (Principal Components - PC)

- PCA tìm trục có phương sai lớn nhất trong tập huấn luyện (Thành phần chính số 1 - PC1).
- Tiếp theo, nó tìm một trục thứ 2 **trực giao (vuông góc)** với trục 1 và chứa nhiều phương sai nhất từ phần còn lại (PC2).
- Quá trình tiếp diễn để tìm PC3, PC4... tối đa bằng số chiều của không gian gốc.
- Trục thứ i được gọi là **thành phần chính thứ i** của dữ liệu.

Toán học PCA: Phân tách giá trị số ít (SVD)

- Để tìm tất cả các thành phần chính, ta sử dụng kỹ thuật phân tách ma trận tiêu chuẩn gọi là **Phân tách giá trị số ít (Singular Value Decomposition - SVD)**.
- SVD phân tách ma trận huấn luyện X thành ba ma trận $U \cdot \Sigma \cdot V^T$.
- Ma trận V^T chứa tất cả các thành phần chính cần tìm (các vector đơn vị trực giao).

- Công thức:
$$V^T = \begin{pmatrix} | & | & \dots & | \\ c_1 & c_2 & \dots & c_n \\ | & | & & | \end{pmatrix}^T$$

Tính toán PCA bằng NumPy (Phần 1)

Dùng hàm `svd()` của NumPy. **Lưu ý quan trọng:** PCA giả định rằng dữ liệu được *căn giữa (centered)* xung quanh gốc tọa độ. Cần trừ đi trung bình trước khi chạy SVD.

```
1 import numpy as np
2
3 # Giả sử X là tập dữ liệu của bạn
4 X_centered = X - X.mean(axis=0)
5
6 # Phân tách giá trị số ít
7 U, s, Vt = np.linalg.svd(X_centered)
8
9 c1 = Vt[0] # Thành phần chính thu 1 (PC1)
10 c2 = Vt[1] # Thành phần chính thu 2 (PC2)
11
```


Chiếu dữ liệu xuống d chiều

- Sau khi có các thành phần chính (PC), ta có thể giảm kích thước không gian bằng cách chiếu tập dữ liệu lên siêu phẳng tạo bởi d thành phần chính đầu tiên ($d < n$).
- Ma trận W_d được định nghĩa là ma trận chứa d cột đầu tiên của V (hay d hàng đầu tiên của V^T chuyển vị).
- Phép chiếu: $X_{d-proj} = X \cdot W_d$

Thực hiện phép chiếu bằng NumPy

Đoạn mã sau chiếu tập huấn luyện lên mặt phẳng 2D định nghĩa bởi hai PC đầu tiên:

```
1 # Chọn 2 thành phần chính đầu tiên và chuyển vi  
2 W2 = Vt[:2].T  
3  
4 # Thực hiện phép nhân ma trận (phép chiếu)  
5 X2D = X_centered @ W2  
6
```

Bây giờ X2D chính là dữ liệu 2D đã được giảm chiều, bảo toàn được tối đa lượng phương sai có thể từ tập X ban đầu.

Sử dụng Scikit-Learn cho PCA

Trong thực tế, Scikit-Learn cung cấp lớp PCA tự động xử lý SVD và quan trọng nhất là **tự động căn giữa dữ liệu**.

```
1 from sklearn.decomposition import PCA
2
3 pca = PCA(n_components=2)
4 X2D = pca.fit_transform(X)
5
6 # Lay cac vector thanh phan chinh
7 print(pca.components_)
8
```

- `pca.components_` chứa các thành phần chính đã tìm thấy.

Tỷ lệ phương sai giải thích (Explained Variance Ratio)

- Đây là một chỉ số quan trọng cho biết tỷ lệ phương sai của tập dữ liệu phân bố dọc theo từng thành phần chính.
- Xem kết quả này qua biến `explained_variance_ratio_`.

Đánh giá tỷ lệ phương sai giải thích

```
1 >>> pca.explained_variance_ratio_  
2 array([0.7578477 , 0.15186921])  
3
```

- Ý nghĩa: $\approx 76\%$ phương sai của dữ liệu nằm dọc theo trục PC1, và $\approx 15\%$ nằm trên trục PC2.
- Tổng cộng PC1 và PC2 giải thích được $> 90\%$ phương sai. Điều này ngụ ý rằng các PC tiếp theo (PC3, PC4...) chứa rất ít thông tin hữu ích (chỉ còn khoảng 9%).

Chọn số chiều phù hợp như thế nào?

- **Quy tắc chung:** Trừ khi giảm xuống 2D hoặc 3D để vẽ biểu đồ trực quan, hãy chọn số chiều d sao cho tổng phương sai được giữ lại đủ lớn (ví dụ: **95%**).
- Nghĩa là, bạn sẽ thêm lần lượt PC1, PC2, PC3... cho đến khi tổng tỷ lệ phương sai giải thích vượt qua mức 95%.

Tìm số chiều để giữ 95% phương sai

Chạy PCA mà không giảm chiều để lấy toàn bộ các PCs, tính toán lũy kế:

```
1 from sklearn.datasets import fetch_openml
2 import numpy as np
3
4 mnist = fetch_openml('mnist_784', as_frame=False)
5 X_train = mnist.data[:60_000]
6
7 pca = PCA()
8 pca.fit(X_train)
9
10 cumsum = np.cumsum(pca.explained_variance_ratio_)
11 d = np.argmax(cumsum >= 0.95) + 1
12 # d bằng 154
13
```

Đặt `n_components` là tỷ lệ phần trăm

Một cách tốt hơn (và nhàn hơn) là truyền thẳng mức độ phương sai bạn muốn bảo toàn dưới dạng một số thập phân từ 0.0 đến 1.0 cho `n_components`:

```
1 # Scikit-learn sẽ tự tính toán số chiều tối thiểu
2 pca = PCA(n_components=0.95)
3
4 X_reduced = pca.fit_transform(X_train)
5
6 print(pca.n_components_)
7 # Output: 154
8
```


Vẽ biểu đồ phương sai giải thích theo số chiều

Một phương pháp trực quan là vẽ mảng `cumsum` (phương sai tích lũy).

(Giả lập Hình 8-8: Biểu đồ đường tăng dần và tạo thành một điểm uốn - elbow point - khi đạt $\approx 100 - 150$ chiều, sau đó đường đi ngang vì các chiều thêm vào không mang thêm phương sai.)

Tại "điểm uốn" (elbow), việc tăng thêm số chiều không mang lại quá nhiều giá trị thông tin, giúp người phân tích dễ dàng quyết định số chiều cần thiết.

Điều chỉnh số chiều như một siêu tham số

- Nếu bạn sử dụng PCA như một bước *tiền xử lý (preprocessing)* cho một tác vụ Machine Learning khác (ví dụ Rừng ngẫu nhiên), bạn có thể coi số chiều d như một **siêu tham số (Hyperparameter)**.
- Có thể dùng `GridSearchCV` hoặc `RandomizedSearchCV` để tìm số chiều tốt nhất cho bài toán đó.

Sử dụng RandomizedSearchCV cho PCA

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3 from sklearn.pipeline import make_pipeline
4
5 clf = make_pipeline(PCA(random_state=42), RandomForestClassifier(random_state
    =42))
6 param_distrib = {
7     "pca__n_components": np.arange(10, 80),
8     "randomforestclassifier__n_estimators": np.arange(50, 500)
9 }
10 rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
    random_state=42)
11 rnd_search.fit(X_train[:1000], y_train[:1000])
12
13 print(rnd_search.best_params_)
14 # pca__n_components tối ưu chỉ là 23 thay vì 154
15
```

PCA dùng để nén dữ liệu (Compression)

- Sau khi áp dụng PCA bảo toàn 95% phương sai, MNIST đã giảm từ 784 đặc trưng xuống còn 154.
- Tập dữ liệu lúc này chỉ còn bằng $\approx 20\%$ kích thước ban đầu, trong khi chỉ **mất 5% thông tin!**
- Mức độ nén tuyệt vời này giúp tiết kiệm bộ nhớ RAM và tăng tốc vượt trội thuật toán học máy phía sau.

Tái tạo dữ liệu (Giải nén)

- Bạn có thể *giải nén* tập dữ liệu 154 chiều ngược trở lại thành 784 chiều.
- Tuy nhiên, đây là nén **có hao hụt (lossy compression)**, bạn sẽ không nhận lại được hoàn toàn 100% pixel gốc (thiếu 5% phương sai đã loại bỏ).
- Hình ảnh tái tạo sẽ bị mờ đi một chút, nhưng phần lớn nội dung gốc vẫn nguyên vẹn.
- *Sai số tái tạo (Reconstruction Error)*: Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu giải nén.

Phép biến đổi ngược (inverse_transform)

Sử dụng `inverse_transform` trên đối tượng PCA:

```
1 X_recovered = pca.inverse_transform(X_reduced)
```

(Giả lập Hình 8-9: Hình ảnh MNIST gốc so sánh với hình ảnh đã qua nén/giải nén bằng PCA. Hình giải nén tuy không sắc nét hoàn hảo bằng nhưng vẫn dễ dàng đọc được là số mấy).

PCA ngẫu nhiên (Randomized PCA)

- Thuật toán **Randomized PCA** dùng kỹ thuật ngẫu nhiên hóa để xấp xỉ tìm d thành phần chính đầu tiên một cách nhanh chóng.
- Độ phức tạp tính toán: $O(m \times d^2) + O(d^3)$, thay vì $O(m \times n^2) + O(n^3)$ của SVD tiêu chuẩn.
- Nó nhanh hơn đáng kể so với PCA đầy đủ khi $d \ll n$ (số chiều giảm xuống nhỏ hơn nhiều số chiều ban đầu).

Cài đặt Randomized PCA

Chỉ định tham số `svd_solver="randomized"`:

```
1 rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
2
3 X_reduced = rnd_pca.fit_transform(X_train)
4
```


PCA tăng dần (Incremental PCA)

- **Vấn đề của PCA thông thường:** Yêu cầu TOÀN BỘ tập dữ liệu phải nằm gọn trong bộ nhớ (RAM) cùng một lúc.
- **Giải pháp:** Incremental PCA (IPCA) cho phép bạn chia tập huấn luyện thành các *mini-batch* và đẩy chúng vào mô hình từng lô nhỏ một.
- Hữu ích cho các tập dữ liệu khổng lồ (Out-of-core learning) và để áp dụng PCA theo thời gian thực (Online).

Cài đặt Incremental PCA với array_split

Lưu ý gọi phương thức `partial_fit()` ở từng bước thay vì `fit()`:

```
1 from sklearn.decomposition import IncrementalPCA
2 import numpy as np
3
4 n_batches = 100
5 inc_pca = IncrementalPCA(n_components=154)
6
7 for X_batch in np.array_split(X_train, n_batches):
8     inc_pca.partial_fit(X_batch)
9
10 # transform áp dụng cho toàn bộ
11 X_reduced = inc_pca.transform(X_train)
12
```

Tối ưu Incremental PCA với tập bộ nhớ (memmap)

Sử dụng `np.memmap` để sử dụng file nhị phân trên đĩa như thể nó nằm trong RAM. Dữ liệu chỉ tải vào RAM khi cần:

```
1 # doc file đã lưu dưới dạng memmap
2 filename = "my_mnist.mmap"
3 X_mmap = np.memmap(filename, dtype="float32", mode="readonly").reshape(-1, 784)
4
5 batch_size = X_mmap.shape[0] // n_batches
6 inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
7
8 # inc_pca hiểu cách hoạt động của memmap và sẽ lấy từng đoạn dữ liệu theo
   batch_size
9 inc_pca.fit(X_mmap)
10
```

Phép chiếu ngẫu nhiên (Random Projection)

- Chiếu dữ liệu xuống không gian thấp bằng một **ma trận ngẫu nhiên**.
- Thay vì dùng SVD phức tạp, Random Projection chứng minh rằng một ma trận chiếu tạo ngẫu nhiên *rất có khả năng bảo toàn khoảng cách* giữa các điểm dữ liệu.
- Ưu điểm: **Không cần huấn luyện!** Chỉ phụ thuộc vào hình dạng ma trận ban đầu. Rất nhẹ và cực nhanh cho dữ liệu chiều cực lớn (hàng chục nghìn features).

- **Định lý Johnson-Lindenstrauss (JL):** Có thể chiếu m điểm từ không gian chiều cao xuống không gian d chiều sao cho khoảng cách bình phương giữa mọi cặp điểm chỉ bị méo mó tối đa là một sai số ε .
- Ví dụ: Bạn có 5.000 điểm (m), với dung sai $\varepsilon = 0.1$.
- Định lý tính được $d \approx 7300$ chiều là đủ an toàn. Phương trình này **không quan tâm đến số chiều gốc n** (ngay cả khi $n = 1$ triệu).

Tìm số chiều tối thiểu bảo toàn khoảng cách

Sử dụng Scikit-Learn để tính toán theo JL lemma:

```
1 from sklearn.random_projection import johnson_lindenstrauss_min_dim
2
3 m = 5000 # so truong hop
4 eps = 0.1 # dung sai
5
6 d = johnson_lindenstrauss_min_dim(m, eps=eps)
7 print(d)
8 # Output: 7300
9
```

Phép chiếu ngẫu nhiên thủ công

Tạo một ma trận P lấy mẫu từ phân phối chuẩn và nhân trực tiếp:

```
1 import numpy as np
2
3 n = 20_000 # so chieu ban dau
4 m = 5_000  # so mau
5 d = 7300   # chieu muc tieu
6
7 # Tao ma tran ngau nhien P
8 P = np.random.randn(d, n) / np.sqrt(d)
9
10 X = np.random.randn(m, n)
11
12 # Thuc hien giam chieu
13 X_reduced = X @ P.T
14
```

Scikit-Learn Random Projections

Dễ dàng hơn với 2 lớp xây dựng sẵn:

```
1 from sklearn.random_projection import GaussianRandomProjection
2 from sklearn.random_projection import SparseRandomProjection
3
4 # 1. Gaussian Random Projection
5 gaussian_rnd_proj = GaussianRandomProjection(eps=0.1, random_state=42)
6 X_reduced = gaussian_rnd_proj.fit_transform(X)
7
8 # 2. Sparse Random Projection
9 sparse_rnd_proj = SparseRandomProjection(eps=0.1)
10 # Hoạt động tương tự nhưng ma trận chiều hình thành chủ yếu bằng số 0.
11 # Giúp tiết kiệm RAM cực mạnh (vd 25MB thay vì 1.2GB)
```


Nhúng tuyến tính cục bộ (Locally Linear Embedding - LLE)

- Là một kỹ thuật **Giảm chiều phi tuyến tính (NLDR)** và thuộc họ **Học đa tạp (Manifold Learning)**.
- Không sử dụng phép chiếu như PCA.
- **Cơ chế:**
 - 1 Đo lường cách mỗi điểm liên kết tuyến tính với các điểm láng giềng gần nhất của nó.
 - 2 Tìm biểu diễn chiều thấp sao cho mỗi liên kết (khoảng cách cục bộ) này được **bảo toàn tốt nhất**.
- Tuyệt vời để mở cuộn các đa tạp bị xoắn (như Swiss roll) nếu dữ liệu không có quá nhiều nhiễu.

Sử dụng LocallyLinearEmbedding cho Swiss roll

```
1 from sklearn.datasets import make_swiss_roll
2 from sklearn.manifold import LocallyLinearEmbedding
3
4 # Tao du lieu Swiss roll 3D
5 X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
6
7 # Giam xuong 2D bang LLE, danh gia tren 10 lang gieng gan nhat
8 lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
9 X_unrolled = lle.fit_transform(X_swiss)
10
```

(Giả lập Hình 8-10: Dữ liệu Swiss Roll 3D được "mở cuộn" thành một mặt phẳng 2D thành công, duy trì quan hệ giữa các điểm bên cạnh nhau).

Thuật toán LLE: Bước 1 (Mô hình hóa cục bộ)

- Đối với mỗi điểm $x^{(i)}$, LLE tìm k láng giềng gần nhất của nó.
- Sau đó cố gắng **tái tạo điểm** $x^{(i)}$ như một hàm tuyến tính (tổng có trọng số) của k láng giềng đó.
- Mục tiêu: Tìm các trọng số $w_{i,j}$ sao cho khoảng cách bình phương giữa $x^{(i)}$ và $\sum w_{i,j}x^{(j)}$ là nhỏ nhất có thể.
- Kết thúc bước 1, ta có một ma trận trọng số W phản ánh chính xác cấu trúc hình học **cục bộ** của tập dữ liệu trong không gian gốc (chiều cao).

Thuật toán LLE: Bước 2 (Giảm chiều bảo toàn khoảng cách)

- Ánh xạ các điểm vào không gian d chiều (với $d < n$).
- Giữ nguyên ma trận trọng số W đã tìm ở Bước 1.
- Tìm các vị trí điểm mới (tọa độ mới) trong không gian d chiều, sao cho chúng **vẫn thỏa mãn ma trận trọng số** đó càng gần càng tốt.
- *Điểm yếu của LLE*: Mở rộng kém cho tập dữ liệu rất lớn.

Các kỹ thuật giảm chiều khác: MDS & Isomap

- **MDS (Multidimensional Scaling):** Cố gắng giảm chiều trong khi bảo toàn **khoảng cách vật lý** giữa các trường hợp như ở không gian gốc. Làm phẳng thay vì mở cuộn.
- **Isomap:** Xây dựng đồ thị nối các láng giềng, sau đó giảm chiều trong khi bảo toàn **khoảng cách trắc địa (geodesic distance)** giữa các điểm. (Trắc địa là đi qua các điểm kết nối, không phải bay đường chim bay). Thích hợp mở cuộn đa tạp.

Các kỹ thuật giảm chiều khác: t-SNE & LDA

- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Giảm chiều bằng cách giữ các điểm tương đồng ở gần nhau, nhưng đẩy các điểm dị biệt ra thật xa. Đặc biệt tốt để **phát hiện cụm trực quan** (như vẽ bản đồ MNIST 2D).
- **LDA (Linear Discriminant Analysis):** Một thuật toán **Phân loại** (Classification). Trong quá trình huấn luyện nó sẽ học trực phân biệt rõ rệt nhất giữa các lớp. Bạn dùng trực này để chiếu dữ liệu, giúp giữ cho các nhãn cách xa nhau nhất có thể → Tiềm xử lý tuyệt vời cho thuật toán phân loại khác.

Tổng kết Chương 8

- **Khái niệm & Động lực:** Giảm số chiều dữ liệu giúp phòng tránh "Lời nguyền số chiều", tăng tốc mô hình và khả năng trực quan hóa.
- **PCA:** Phương pháp Phép chiếu, sử dụng SVD, là kỹ thuật mặc định ưu tiên số một. Nắm vững Incremental PCA (cho dữ liệu lớn) và Randomized PCA.
- **Random Projection:** Vô địch về tốc độ và tiết kiệm RAM cho dữ liệu siêu lớn mà không cần huấn luyện.
- **Manifold Learning (LLE, Isomap...):** Có khả năng "mở cuộn" các hình khối xoắn ốc phức tạp, thích hợp cho dữ liệu có cấu trúc đa tạp phi tuyến tính.

CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE!

Q & A